Weighted Interval Scheduling

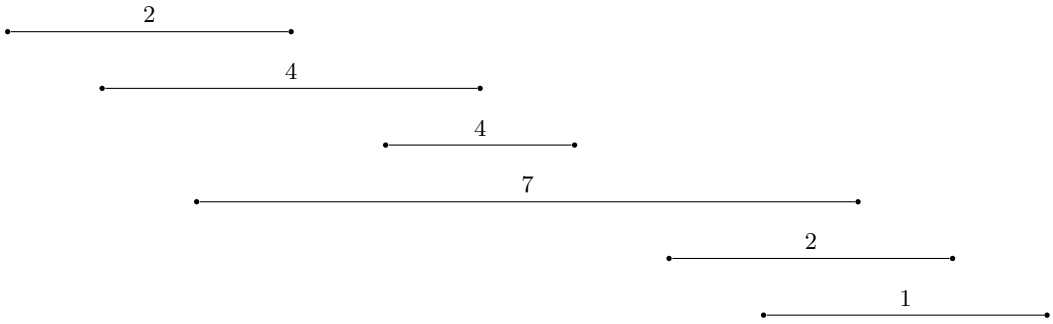
Warm-Up: we are given a set of n intervals, numbered $1 \dots n$, each of which has a start time s_i and a finish time f_i. For each interval, we want to compute a value $p[i]$, which is the interval j with the *latest* finish time f_j such that $f_j \leq s_i$; that is, the last-ending interval that finishes before interval i starts. If no intervals end before interval i begins, then $p[i] = 0$.

Give an $\mathcal{O}(n \log n)$ time algorithm that computes $p[i]$ for all intervals. You may assume that the intervals are already sorted by finish time.

The Big Problem: Fed up after the first midterm in CompSci 161, your friend has decided to change majors to one that grades based only on attendance. The only question is which classes your friend should take in Fall quarter. The classes all meet once a day, at different times and lengths, and are worth different amounts of credits. Your friend's goal is to maximize the amount of credits earned in that quarter without having to skip any classes (as this may interfere with passing those classes).

Problem Statement: More formally, we are given a set of n intervals, each of which has a start time s_i, a finish time f_i, and a value v_i. Our goal is to select a subset of the intervals such that no two selected intervals overlap and the total value of those taken is maximized.

Example Input: Please be aware that sample input will not always be provided in CompSci 161; one of the educational objectives is for you to be able to solve a problem in the abstract.



© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Let's solve this *recursively* (yay!). We will write a function `WIS(`i`)` that returns the optimal number of credits obtainable among intervals (classes) $1 \dots i$. We can then call `WIS(n)` to figure out the optimal number of credits obtainable among all intervals.

A key observation here is that your friend will either take class i or your friend won't take class i.

```
WIS(i)

    // Base Case:



   // If my friend doesn't take class i:

   value_if_not_taken =


   // If my friend takes class i:

   value_if_taken =


   //return something:
```

Should we implement it that way? We now have a recursive solution. Think back to the Fibonacci example earlier in lecture. What will happen if we implement this program this way?

We can now move to have a solution that uses no recursive *function calls*. Note that our solution is still conceptually recursive. The iterative solution will also allow us to output which courses to take, not simply the optimal value.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Homework Assignments (not for you)

Professor Shindler gives lots of homework assignments, each of which have an easy version and a hard version[1]. Each student is allowed, for each homework, to submit either their answer to the easy version (and get $e_i > 0$ points) or the hard version (and get $h_i > 0$ points, which is also guaranteed to always be more than e_i) or to submit neither (and get 0 points for the assignment). Note that e_i might have different values for each i, as might h_i. The values for all n assignments are known at the start of the quarter.

The catch is that the hard version is, perhaps not surprisingly, more difficult than the easy version. In order for you to do the hard version, you must have not done the immediate previous assignment at all: neither the easy nor the hard version (and thus are more relaxed, de-stressed, etc). You are allowed to do the hard version of assignment one if you want. Your goal is to maximize the number of points you get from homework assignments over the course of the quarter. Give an efficient dynamic programming algorithm to determine the largest number of points possible for a given quarter's homework choices.

Meal Planning

College students get a lot of free food at various events. Suppose you have a schedule of the next n days marked with those days when you get a free dinner, and those days on which you must acquire dinner on your own. On any given day you can buy dinner at the cafeteria for \$6. Alternatively, you can purchase one week's groceries for \$20, which will provide dinner for each day that week. However, because you don't have a fridge, the groceries will go bad after seven days (including the day of purchase) and any leftovers must be discarded. Due to your very busy schedule, these are your only three Foodions for dinner each night.

Give an efficient algorithm to determine, given the schedule of free meals, the minimum amount of money you must spend to make sure you have dinner each night.

Hint: Start by writing a recursive procedure to determine how you should eat dinner on the last night.

[1] This is not the actual policy.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Longest Common Subsequence

Reading: G/T §12.5. **Problem Statement**: A *subsequence* of a given sequence is just the given sequence with zero or more elements left out. Given two sequences X and Y, we say that a sequence Z is a *common subsequence* of X and Y if Z is a subsequence of both X and Y. Our goal is to find the maximum length common subsequence.

Examples:

X	Y	LCS
complete	continue	cote
exercise	determine	eerie
surface	character	race
toward	thousand	toad

As with the previous lecture, let's determine the general recursive solution first. Can you determine something tautological about the LCS of sequences X and Y? Determine the recursive and iterative algorithms for LCS.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Edit Distance

Reading: Erickson, §3.7

The **Edit distance** problem is as follows. We are given two strings (not necessarily of equal length). We want to convert the first string to the other by a sequence of insertions, deletions, and substitutions. The **cost** is the number of operations we perform.

For example, if we want to convert FOOD to MONEY, we could do this:
FOOD $\rightarrow$ MOOD $\rightarrow$ MOND $\rightarrow$ MONED $\rightarrow$ MONEY

One way to visualize this is by alignment:

F	O	O		D
M	O	N	E	Y

We **define** Edit(i, j) to be the **minimum cost** to convert $X[1 \ldots i]$ to $Y[1 \ldots j]$.

What happened in the last column?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Longest Increasing Subsequence

Reading: Erickson, §3.6

Problem Statement: Recall that a *subsequence* of a given sequence is just the given sequence with zero or more elements left out. The goal in this problem is to find the longest *increasing* subsequence of the given input. For example, if our input is:

30	10	20	40	60	50	70	90	80

Then our longest increasing subsequence is of length six; one such is 10, 20, 40, 50, 70, 80.

Initial Algorithm

Suppose $A[0] = -\infty$ (or equivalent for the type of data stored in the input sequence). Let's write LISBigger(i, j), which will be the longest increasing subsequence of $A[j \dots n]$ in which every element is larger than $A[i]$.

What is our top-level call to LISBigger, if we think of this as a recursively implemented solution? Equivalently, in which element of the memoization table does the solution reside?

Improved Algorithm

Suppose $A[0] = -\infty$ (or equivalent for the type of data stored in the input sequence). Let's write LISfirst(i), which will be the length of the longest increasing subsequence of $A[i \dots n]$ that begins with $A[i]$.

What is our top-level call to LISfirst, if we think of this as a recursively implemented solution? Equivalently, in which element of the memoization table does the solution reside?

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.